

الموضوع الثامن | أمثلة تزامن (تزامن ويندوز)

تزامن ويندوز

- يستخدم أقنعة المقاطعة لحماية الوصول إلى الموارد العالمية في الأنظمة أحادية المعالج.
- يستخدم الأقفال الدوارة في الأنظمة متعددة المعالجات. خيط القفل الدوار لن يتم تعطيله أبداً.
- يوفر أيضاً كائنات الموزع في مستوى المستخدم التي قد تعمل كأقفال الاستبعاد المتبادل والسيمافورات والأحداث والمؤقتات.

• الأحداث

الحدث يعمل بشكل مشابه لمتغير الحالة.

- المؤقتات تُنبه خيطاً أو أكثر عندما ينقضي الوقت.
- كائنات الموزع إما في حالة إشعار (الكائن متاح) أو في حالة غير إشعار (الخيط سيحظر).

تزامن لينيكس

لينيكس:

- قبل إصدار النواة 2.6، كان يتم تعطيل المقاطعات لتنفيذ أقسام حاسمة قصيرة.
- من الإصدار 2.6 وما بعده، أصبح لينيكس يدعم التحويل التلقائي بالكامل.
- يوفر لينيكس:
 - السيمافورات (Semaphores).
 - الأعداد الذرية (Atomic integers).
 - الأقفال الدوارة (Spinlocks).
 - نسخ القراءة والكتابة لكل منهما.
- على النظام ذو المعالج الأحادي، تم استبدال الأقفال الدوارة بتمكين وتعطيل التحويل التلقائي للنواة.



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432



مفاهيم أنظمة التشغيل :

OSC :

:



:

الخيوط

في نهاية هذا الدرس، يتوقع من المتعلم أن يكون قادرًا على أن:

✓ تتعرف على مفهوم الـ "خيوط"

✓ تتعرف على برمجة الأنظمة متعددة النواة

✓ تفرّق بين العمليات أحادية الخيوط ومتعددة الخيوط

✓ تفرّق بين خيوط المستخدم وخيوط النواة

✓ تتعرف على النماذج متعددة الخيوط

✓ تتعرف على مكتبات الخيوط (Thread Libraries)

✓ تتعرف على أمثلة على أنظمة التشغيل



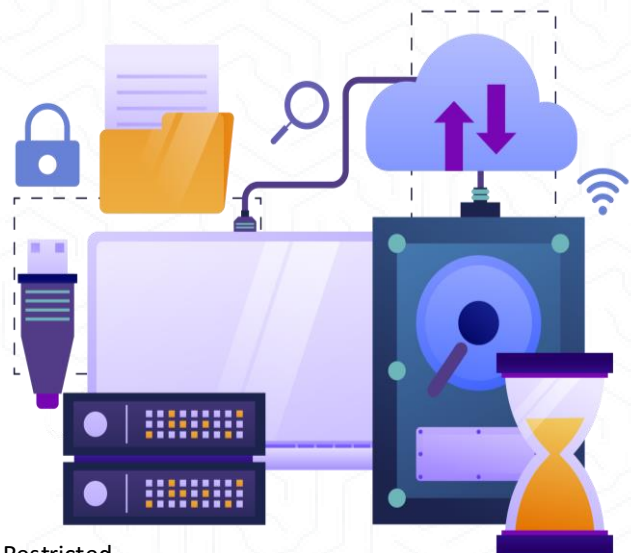


في هذا الدرس سنتناول:

- مفهوم الخيوط.
- الفوائد.
- برمجة متعددة النواة.
- التزامن مقابل التوازي.
- العمليات أحادية الخيط ومتعددة الخيوط.
- خيوط المستخدم وخيوط النواة.
- نماذج البرمجة متعددة الخيوط.
- مكتبات الخيوط في Pthreads و Windows و Java.
- أمثلة على أنظمة التشغيل

مفهوم الخيطوط (Threads)

- الخيط هو أصغر وحدة تنفيذ داخل وحدة المعالجة المركزية وهو هو الوحدة الأساسية لاستخدام وحدة المعالجة المركزية.
- يتكون الخيط من معرف الخيط (Thread ID)، عداد البرنامج (Program Counter)، مجموعة السجلات (Registers)، والمكدس (Stack). يشارك الخيط مع الخيطوط الأخرى في نفس العملية: قسم الكود، وقسم البيانات، والملفات المفتوحة، والإشارات.
- في العمليات التقليدية (أحادية الخيط)، يوجد خيط واحد فقط مسؤول عن تنفيذ جميع المهام.
- في العمليات متعددة الخيطوط، يمكن وجود أكثر من خيط يعمل بشكل متزامن لتنفيذ مهام متعددة ضمن نفس العملية، مما يعزز الأداء والكفاءة.



الموضوع السابع | الخيوط Threads

الدوافع لاستخدام الخيوط (Threads)

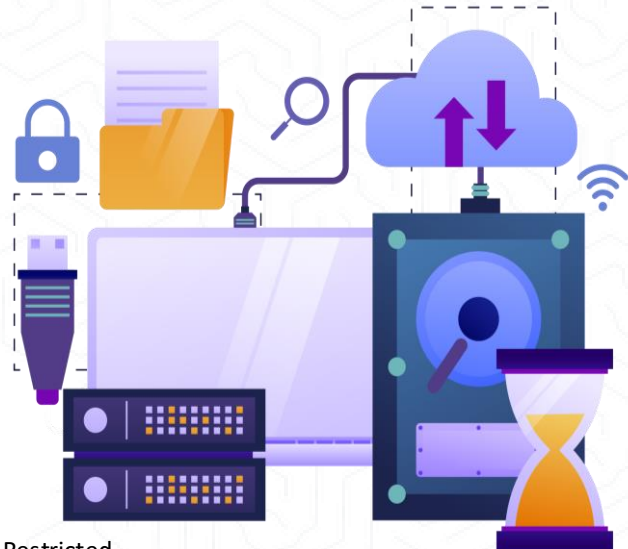
- تعتمد التطبيقات الحديثة بشكل كبير على استخدام الخيوط المتعددة (Multithreading) لتحقيق الأداء العالي والاستجابة السريعة.
- يُمكن تنفيذ المهام المتعددة داخل نفس العملية باستخدام خيوط منفصلة لكل مهمة. ومن أبرز الأمثلة:
 - تحديث العرض (الشاشة).
 - جلب البيانات من قاعدة البيانات أو الإنترنت.
 - تدقيق الإدخال أثناء الكتابة.
 - التعامل مع طلبات الشبكة.

توفير الموارد: إنشاء عملية جديدة يستهلك موارد أكثر مقارنة بإنشاء خيط.

تبسيط التصميم البرمجي: يسهل تقسيم الكود وتحسين الصيانة.

تحقيق الكفاءة: تنفيذ مهام متعددة بشكل متزامن يؤدي إلى تقليل زمن الانتظار وزيادة سرعة التنفيذ.

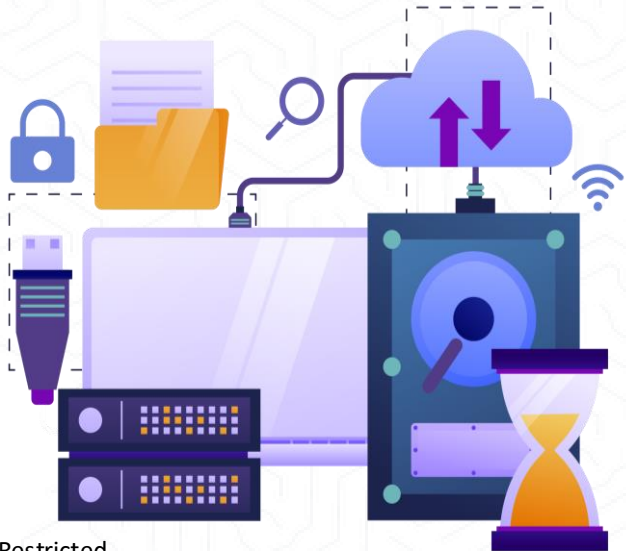
دعم الأنوية المتعددة: أنظمة التشغيل الحديثة غالبًا ما تدعم نواة متعددة الخيوط (Multithreaded Kernel)



الموضوع السابع | الخيوط Threads

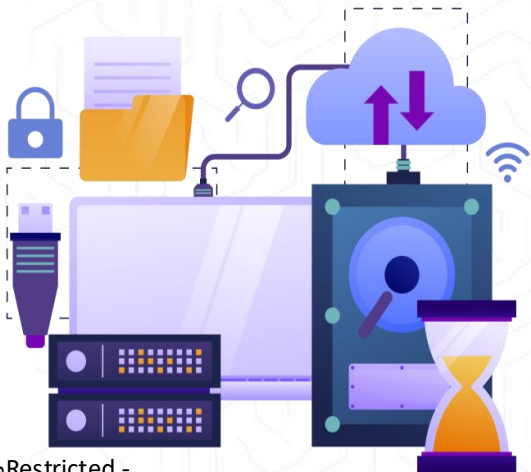
الدوافع لاستخدام الخيوط (Threads)

- استخدام الخيوط يوفر عدة مزايا مهمة:
- توفير الموارد: إنشاء عملية جديدة يستهلك موارد أكثر مقارنة بإنشاء خيط.
- تبسيط التصميم البرمجي: يسهل تقسيم الكود وتحسين الصيانة.
- تحقيق الكفاءة: تنفيذ مهام متعددة بشكل متزامن يؤدي إلى تقليل زمن الانتظار وزيادة سرعة التنفيذ.
- دعم الأنوية المتعددة: أنظمة التشغيل الحديثة غالبًا ما تدعم نواة متعددة الخيوط (Multithreaded Kernel)



الفوائد:

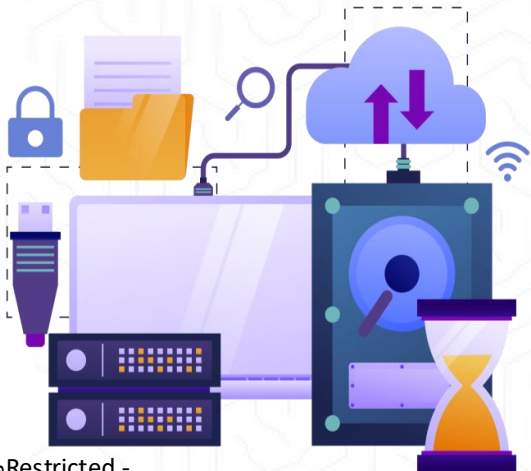
- الاستجابة: تتيح الخيوط الاستمرار في تنفيذ باقي أجزاء العملية حتى في حال توقف أحدها مؤقتًا، مما يحسّن من تجربة المستخدم، خصوصًا في التطبيقات ذات الواجهات الرسومية.
- مشاركة الموارد: تشترك الخيوط في موارد العملية مثل الذاكرة، الملفات المفتوحة، والإشارات، مما يسهل عملية تبادل البيانات بينها دون الحاجة لتقنيات تواصل معقدة.
- التكلفة: إنشاء خيط جديد أقل تكلفة من إنشاء عملية جديدة، لأنه لا يتطلب تخصيص موارد مستقلة بالكامل.
- القابلية للتوسع: تُمكن الخيوط من الاستفادة من المعالجات المتعددة (Multiprocessor Architectures) من خلال تنفيذ المهام بالتوازي، مما يرفع من كفاءة الأداء الكلي للنظام.



الموضوع السابع | برمجة الأنظمة متعددة النواة

برمجة الأنظمة متعددة النواة

- الأنظمة متعددة النواة أو المعالجات المتعددة تفرض العديد من التحديات على المبرمجين، ومن أبرز هذه التحديات:
 - تقسيم الأنشطة: تحديد المهام التي يمكن تنفيذها بالتوازي.
 - تقسيم البيانات: ضمان توزيع البيانات بشكل يحقق الكفاءة ولا يؤدي إلى تضارب.
 - اعتماد البيانات: التأكد من صحة البيانات بين الخيوط المختلفة.
 - الاختبار وتصحيح الأخطاء: يصبح معقدًا نتيجة التزامن بين الخيوط.
- مفاهيم أساسية:
 - التوازي (Parallelism) قدرة النظام على تنفيذ أكثر من مهمة في وقت واحد فعليًا.
 - التزامن (Concurrency) دعم تنفيذ مهام متعددة تبدو وكأنها تنفذ في الوقت نفسه، مع ضمان تنظيم التقدم بينها.

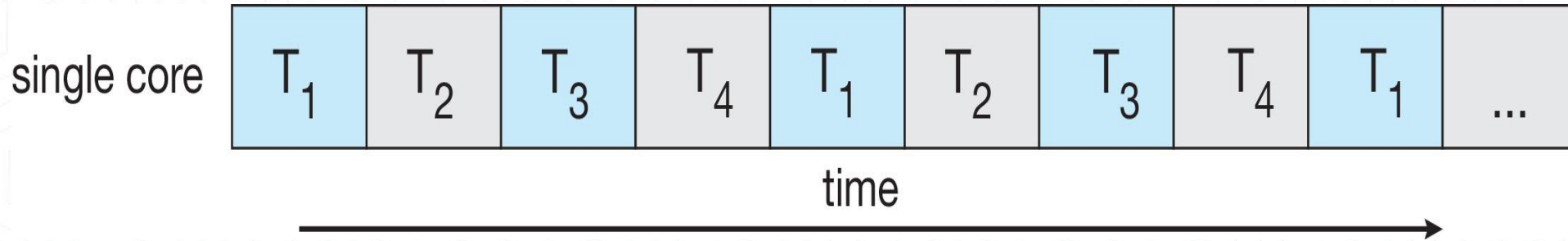


الموضوع السابع | برمجة الأنظمة متعددة النواة

أنواع التوازي:

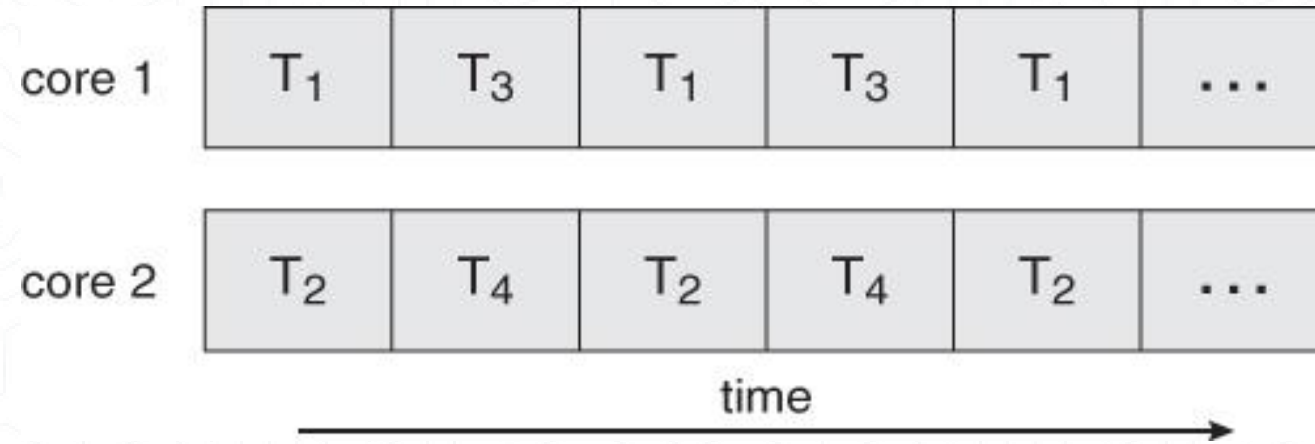
- التوازي البياني (Data Parallelism)
 - يتم تقسيم البيانات الكبيرة إلى أجزاء أصغر وتوزيعها على الأنوية المختلفة، بحيث تقوم كل نواة بتنفيذ نفس التعليمات على جزء مختلف من البيانات.
 - مثال: إذا كان لدينا مصفوفة من القيم، يمكن لكل نواة حساب الجذر التربيعي لجزء منها في نفس الوقت
- التوازي المهام (Task Parallelism)
 - يتم توزيع الخيوط عبر النواة المختلفة، بحيث يؤدي كل خيط عملية مختلفة ومستقلة.
 - مثال: يمكن لخيط أن يتعامل مع إدخال المستخدم، وآخر يعالج البيانات، وثالث يعرض النتائج على الشاشة
 - مع زيادة عدد الخيوط في النظام، يزداد أيضًا الدعم المعماري لهذه الخيوط، حيث تحتوي المعالجات على أنوية متعددة، ويمكن لكل نواة أن تدير عدة خيوط في نفس الوقت.
 - على سبيل المثال، معالج Oracle SPARC T4 يحتوي على 8 أنوية، وكل نواة تدير 8 خيوط مادية، مما يعني أن المعالج يمكنه تشغيل 64 خيطًا في وقت واحد.

التنفيذ المتزامن على نظام ذو نواة واحدة

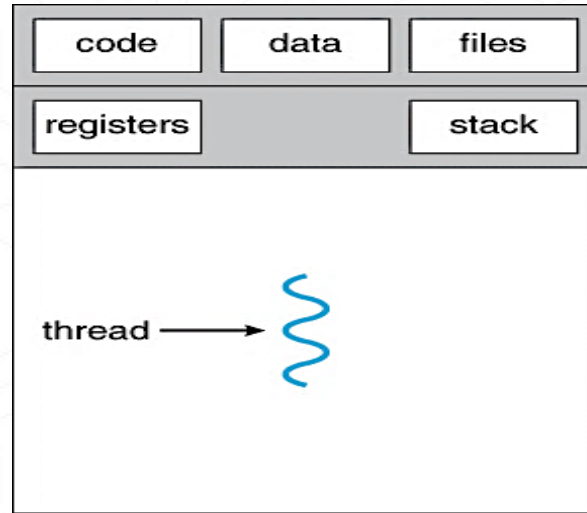


في نظام ذو نواة واحدة، يتم تنفيذ المهام (Threads) بطريقة تشاركية، حيث تقوم النواة بتبديل التنفيذ بين المهام بسرعة كبيرة، مما يعطي انطباعاً بأن المهام تُنفذ في نفس الوقت، لكنها فعلياً تُنفذ بالتتابع. هذه الطريقة مفيدة عندما يكون هناك أكثر من مهمة تحتاج إلى الاستجابة دون الحاجة إلى تنفيذها فعلياً في نفس اللحظة.

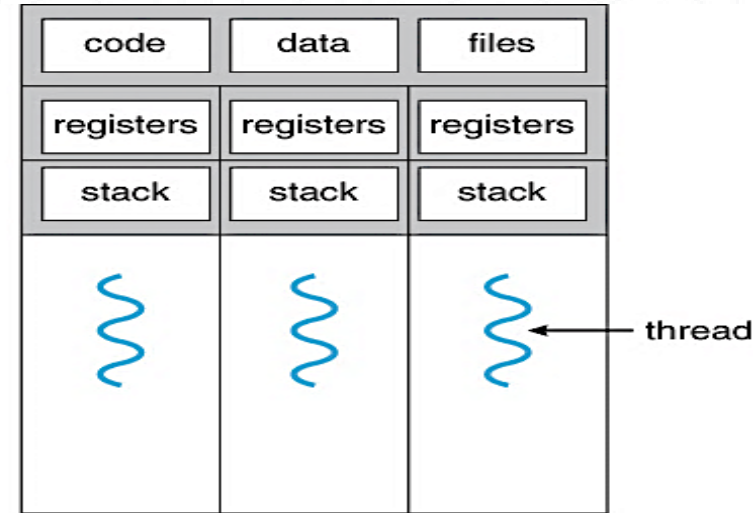
التوازي على نظام متعدد النواة



في نظام متعدد الأنوية، يمكن تنفيذ أكثر من مهمة فعلاً في نفس الوقت، بحيث تُخصص كل نواة لمهمة مختلفة، مما يسرّع الإنجاز ويزيد من الكفاءة، ويعد مثاليًا لتطبيقات تتطلب معالجة مكثفة أو زمن استجابة سريع.



single-threaded process



multithreaded process

- العملية أحادية الخيط: تتكون من خيط واحد فقط ينفذ جميع المهام بالتتابع. تشترك كل المهام في نفس الذاكرة، وتفتقر إلى المرونة عند تنفيذ أكثر من مهمة في وقت واحد
- العمليات متعددة الخيوط: تحتوي على عدة خيوط تعمل بشكل متزامن. جميع الخيوط تشترك في نفس الكود والبيانات والملفات المفتوحة، لكنها تمتلك مكدرات (Stacks) وسجلات (Registers) مستقلة. هذا الأسلوب يزيد من كفاءة الأداء ويُستخدم في التطبيقات التي تتطلب تعدد المهام.

- خيوط المستخدم:
- تتم إدارة الخيوط بواسطة مكتبة خيوط على مستوى المستخدم.
- يوجد ثلاث مكتبات خيوط رئيسية:
 - POSIX Pthreads
 - خيوط Windows
 - خيوط Java
- نظام التشغيل لا يكون على علم مباشر بها، مما يجعل إدارتها أسرع وأبسط، ولكن في حالة حدوث حظر (block) في أحد الخيوط، تتأثر العملية بالكامل.
- خيوط النواة :
- يتم دعمها من قبل نظام التشغيل نفسه. لكل خيط إدارة مستقلة بواسطة النواة، مما يسمح بتنفيذ فعلي متوازي على المعالجات متعددة النوى. على الرغم من كفاءتها العالية، إلا أن إدارتها تتطلب تكلفة أعلى من ناحية الأداء.
- أمثلة: تقريبًا جميع أنظمة التشغيل العامة، بما في ذلك: Windows , Solaris, Linux, Mac OS X

نماذج البرمجة متعددة الخيوط:

- من العديد إلى واحد (Many-to-One).
- من واحد إلى واحد (One-to-One).
- من العديد إلى العديد (Many-to-Many).

Many-to-One من العديد إلى واحد

الموضوع السابع

من العديد إلى واحد (Many-to-One)

- في هذا النموذج، يتم ربط عدة خيوط مستخدم بخيط نواة واحد فقط، ما يعني أن نظام التشغيل لا يرى سوى خيط نواة واحد، مهما بلغ عدد خيوط المستخدم.

- إذا تم حظر أحد خيوط المستخدم، تتوقف جميع الخيوط المرتبطة به، نظرًا لأنها تشترك في نفس خيط النواة.

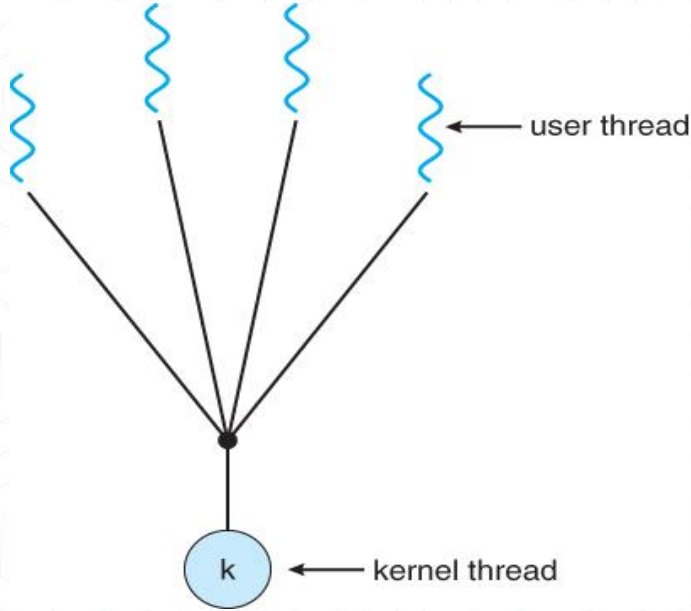
- لا يمكن تنفيذ خيوط متعددة بالتوازي على أنظمة متعددة النواة لأن النواة تتعامل مع خيط واحد فقط، مما يقلل من كفاءة المعالجة في البيئات المتوازية.

- الاستخدام الفعلي: بسبب هذه القيود، نادرًا ما يُستخدم هذا النموذج في الأنظمة الحديثة.

- أمثلة على الأنظمة التي تدعمه

- Solaris Green Threads

- GNU Portable Threads



One-to-One من واحد إلى واحد

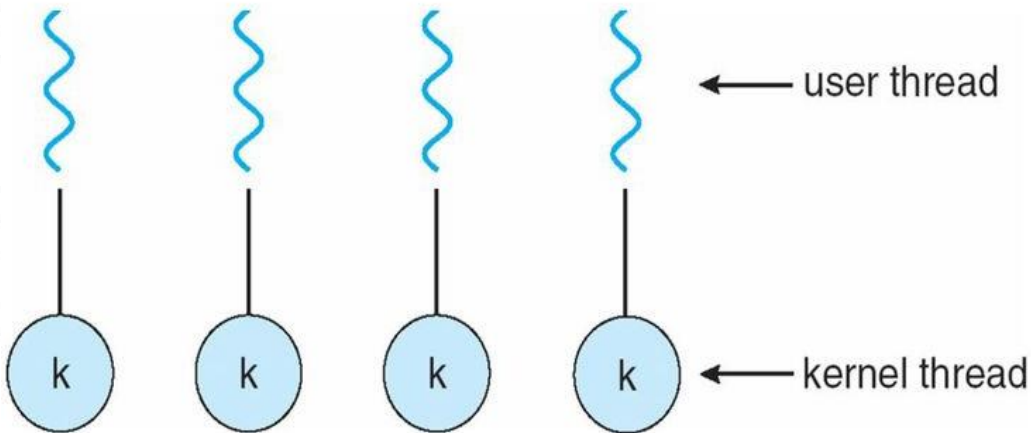
الموضوع السابع

من واحد إلى واحد (One-to-One)

- يتم إنشاء خيط نواة لكل خيط مستخدم، ما يعني أن كل عملية متعددة الخيوط تحتوي على عدد مساوٍ من الخيوط في النواة.
- هذا النموذج يمنح أداءً عاليًا من حيث التوازي، حيث يمكن تنفيذ الخيوط في وقت واحد فعليًا على معالجات متعددة.
- توفر توازي أكبر من نموذج من العديد إلى واحد.
- هذا النموذج قد يكون مكلفًا في الاستخدام المكثف للخيوط، لأن إنشاء خيط نواة جديد يستهلك وقتًا وموارد. ولهذا السبب قد تحد بعض أنظمة التشغيل من عدد الخيوط لكل عملية لتجنب استنزاف الموارد.
- يعد هذا النموذج شائعًا في الأنظمة الحديثة التي تحتاج إلى أداء مرتفع وتطبيقات ذات طبيعة متعددة المهام.

• أمثلة:

- Windows
- Linux
- Solaris 9 وما بعده.



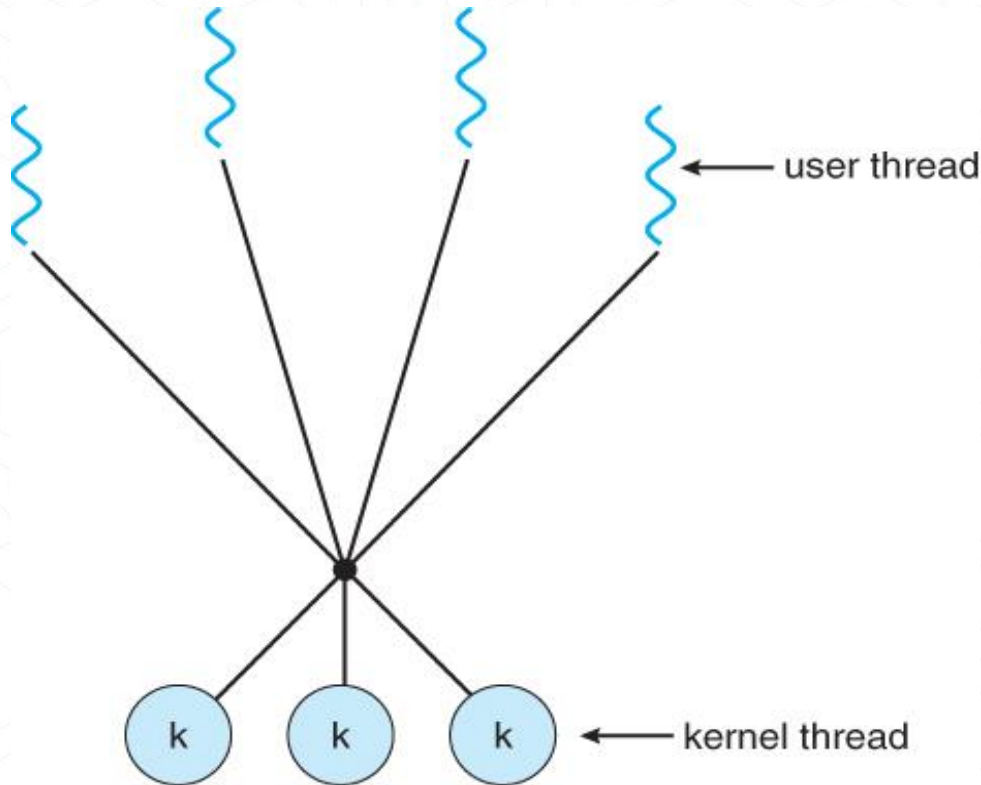
الموضوع السابع | من العديد إلى العديد Many-to-Many

نموذج من العديد إلى العديد (Many-to-Many)

- يعتبر من أكثر النماذج مرونة وكفاءة. يسمح هذا النموذج بربط عدة خيوط مستخدم بعدة خيوط نواة، مما يعني أن كل خيط مستخدم يمكن أن يتم تعيينه لأي خيط نواة متاح عند الحاجة. يتيح للنظام التشغيلي إنشاء عدد كافٍ من خيوط النواة.
- هذا يعطي النظام القدرة على جدولته وتنفيذ المهام بكفاءة حسب الحاجة والموارد المتاحة.
- يمكن للنظام إنشاء عدد كبير من خيوط المستخدم دون التأثير المباشر على موارد النظام.
- مناسب للتطبيقات المعقدة التي تتطلب أداءً عالياً وتوازياً في التنفيذ.
- أمثلة:

• Solaris قبل الإصدار 9

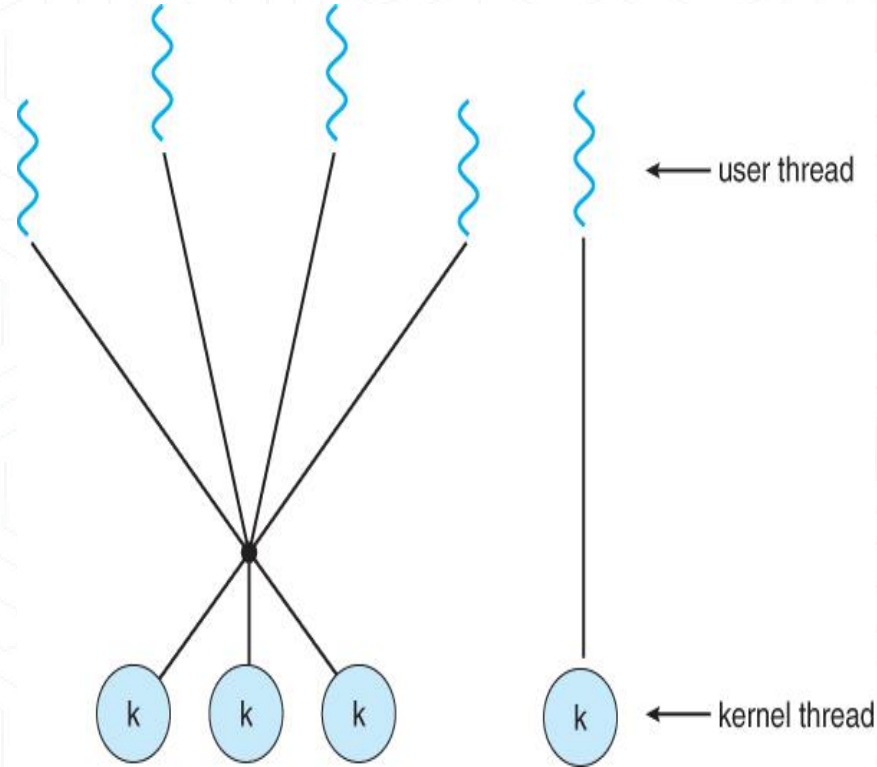
• Windows مع حزمة ThreadFiber



الموضوع السابع | نموذج ذو المستويين (Two-level Model)

نموذج ذو المستويين (Two-level Model)

- نموذج ذو المستويين يجمع بين مميزات النموذجين السابقين (من العديد إلى العديد، ومن العديد إلى واحد). في هذا النموذج:
- تُدار خيوط المستخدم على مستوى المستخدم من خلال مكتبات خاصة.
- يتم ربط بعض خيوط المستخدم مباشرة مع خيوط النواة، مما يمنحها إمكانية تنفيذ متوازي فعلي.
- بينما تُربط خيوط أخرى بعدة خيوط نواة مشتركة، مثل نموذج من العديد إلى العديد.
- هذا النموذج يعطي مرونة عالية، حيث يسمح بوجود خيوط لها أولوية أعلى ترتبط مباشرة مع خيط نواة، مما يحسن الأداء في التطبيقات الحساسة للزمن.
- أمثلة: Solaris 8 , Tru64 UNIX , HP-UX , IRIX والإصدارات الأقدم



مكتبات الخيوط

- بهدف تسهيل إنشاء وإدارة الخيوط (Threads)، توفر مكتبات الخيوط واجهات برمجية (API) جاهزة للمبرمجين.
- تنقسم هذه المكتبات إلى نوعين أساسيين:
 - مكتبة على مستوى المستخدم (User-Level Library) وهي مكتبات تعمل في مساحة المستخدم ولا تحتاج إلى تدخل مباشر من نواة النظام، ما يجعلها أسرع من حيث الأداء لكنها لا تستفيد من ميزات تعدد المعالجات.
 - مكتبة على مستوى النواة (Kernel-Level Library) والتي تتم إدارتها بالكامل بواسطة نظام التشغيل وتوفر دعماً أفضل للتوازي الحقيقي على الأنظمة متعددة النواة، وإن كانت أقل كفاءة من حيث الأداء في بعض الحالات بسبب استدعاءات النظام المتكررة.

خيوط POSIX (Pthreads)

- تُعد خيوط POSIX أو (Pthreads) معيارًا صناعيًا لإنشاء وإدارة الخيوط في أنظمة التشغيل الشبيهة بـ UNIX، وتوفر وسيلة موحدة للمبرمجين للعمل مع الخيوط بغض النظر عن النظام المستخدم.
- وتنفَّذ إما كمكتبة تعمل في مساحة المستخدم (User Space)، أو ضمن النواة (Kernel Space) حسب تصميم النظام.
- تعتمد على معيار POSIX (IEEE 1003.1c) الذي يحدد واجهة برمجية (API) لإنشاء الخيوط ومزامنتها.
- خيوط التنفيذ هي مواصفة قياسية تُستخدم في أنظمة UNIX لإنشاء وإدارة الخيوط، توفر واجهة موحدة للمطورين، بينما تترك كيفية تنفيذ هذه الواجهة إلى المكتبات البرمجية الخاصة بالنظام.
- تُستخدم بشكل شائع في أنظمة التشغيل المستندة إلى UNIX مثل:

- Solaris
- Linux
- Mac OS X

خيوط جافا

- خيوط جافا تُدار بواسطة آلة جافا الافتراضية ((JVM، والتي تُشغل برامج Java بشكل مستقل عن نظام التشغيل.
- تعتمد JVM عادةً على نموذج خيوط نظام التشغيل الأساسي لتوفير الأداء والكفاءة.
- تُستخدم خيوط Java لتنفيذ عمليات متعددة في آنٍ واحد (multi-threading) مما يُحسن استجابة البرنامج.
- يمكن إنشاء خيوط Java عن طريق:
 - تمديد الفئة Thread:
 - يتم إنشاء صنف جديد يرث من الفئة Thread ويُعاد تعريف الدالة run()
 - تنفيذ الواجهة Runnable:
 - يتم إنشاء كائن من صنف يُنفذ واجهة Runnable ويُمرر إلى كائن من نوع Thread

```
public interface Runnable {  
  
    public abstract void run();  
  
}
```

أمثلة على أنظمة التشغيل:

- خيوط ويندوز Windows Threads.
- خيوط لينكس Linux Threads.

الموضوع السابع | خيوط ويندوز Windows Threads

خيوط ويندوز

- تعتمد خيوط ويندوز على واجهة برمجة التطبيقات الخاصة بويندوز (Windows API)، وهي الواجهة الرئيسية لإنشاء الخيوط وإدارتها في أنظمة مثل ويندوز 98، ويندوز NT ، ويندوز 2000، ويندوز XP ، و ويندوز 7.
- يتم استخدام نموذج واحد إلى واحد (one-to-one) حيث يتم ربط كل خيط مستخدم بخيط نواة (kernel-level).
- كل خيط يحتوي على:
 - معرف الخيط (Thread ID): رقم فريد يميّز كل خيط.
 - مجموعة السجلات التي تمثّل الحالة الحالية للمعالج أثناء تنفيذ الخيط
 - مكدرات منفصلة (أحدهما للمستخدم والآخر للنواة) لاستخدامها عندما يعمل الخيط في وضع المستخدم أو وضع النواة.
 - منطقة بيانات محلية: تُستخدم لتخزين بيانات مؤقتة خاصة بالخيط، وغالبًا ما تُستخدم من قبل المكتبات الديناميكية (DLLs)
 - سياق الخيط (Thread Context) يشمل كل ما سبق (السجلات، المكدرات، البيانات المؤقتة)، ويُستخدم عند حفظ واستعادة حالة الخيط.

الموضوع السابع | خيوط ويندوز Windows Threads

خيوط ويندوز

الهياكل الأساسية لخيوط ويندوز تشمل:

1. كتلة بيانات تنفيذية :

- تحتوي على مؤشر إلى العملية التي ينتمي إليها الخيط ومؤشر إلى كتلة بيانات خيط النواة.
- تُخزَّن في مساحة النواة.

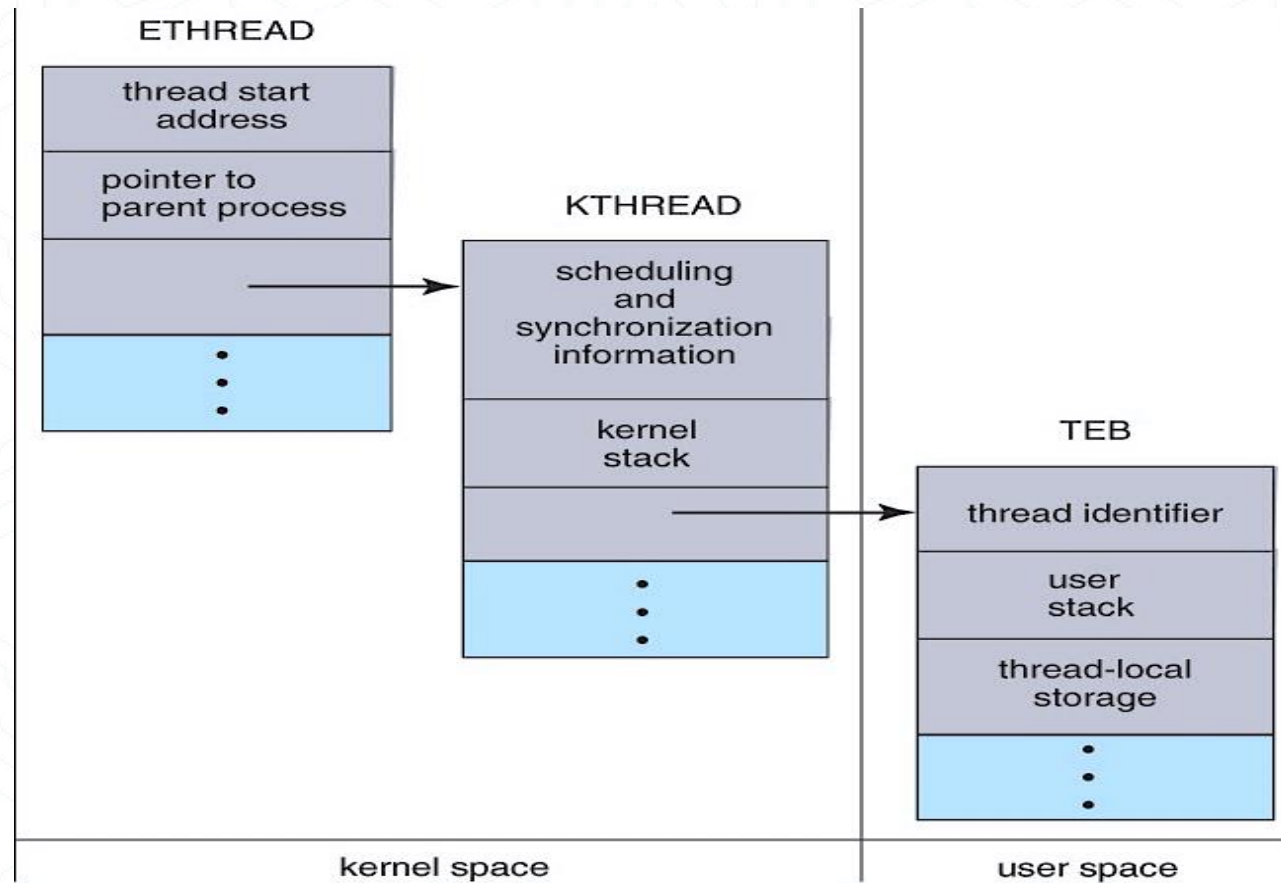
2. كتلة بيانات خيط النواة:

- تحتوي على معلومات الجدولة والمزامنة، مكس وضع النواة، ومؤشر إلى كتلة بيئة الخيط.
- تُخزَّن في مساحة النواة.

3. كتلة بيئة الخيط:

- تحتوي على معرف الخيط، مكس وضع المستخدم، وتخزين خاص بالخيط.
- تُخزَّن في مساحة المستخدم.

Windows Threads Data Structures



خيوط لينكس

- يشير نظام لينكس إلى الخيوط باسم المهام (tasks) بدلاً من استخدام مصطلح الخيوط.
- يتم إنشاء الخيوط باستخدام نداء النظام clone، الذي يُتيح إنشاء مهام فرعية.
- المهام الفرعية (الخيوط) يمكنها مشاركة نفس مساحة العناوين مع المهمة الأصلية (العملية).
- يتم التحكم في سلوك المهام الجديدة باستخدام العلامات (Flags) التي تُمرر أثناء استدعاء clone، ومنها:

flag	
CLONE_FS	تشارك المهمة الجديدة إعدادات النظام المتعلقة بالملفات (مثل دليل العمل الحالي).
CLONE_VM	تشارك نفس مساحة الذاكرة الافتراضية مع العملية الأصلية.
CLONE_SIGHAND	تشارك معالجات الإشارات مع العملية الأصلية.
CLONE_FILES	تشارك الملفات المفتوحة مع العملية الأصلية.

- يُستخدم struct task_struct لتمثيل كل مهمة (أو خيط) في النظام، ويحتوي هذا الهيكل على بيانات العملية سواء كانت مستقلة أو مشتركة.



الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432



دبلوم امن المعلومات

اسم المقرر: : مفاهيم انظمة التشغيل

رمز المقرر: OSC

الموضوع: السادس



الموضوع السادس إدارة أجهزة الإدخال/الإخراج و إدارة الملفات

الأهداف

في نهاية هذا الدرس، يتوقع من المتعلم أن يكون قادرًا على أن:

- ✓ تتعرف على إدارة أجهزة الإدخال/الإخراج
- ✓ تتعرف على وظيفة وحدة التحكم في الأجهزة Device Controller
- ✓ تتعرف على الوصول المباشر للذاكرة
- ✓ تتعرف على إدارة الملفات
- ✓ تتعرف على الخصائص الأساسية للملفات
- ✓ تتعرف على العمليات الأساسية على الملفات
- ✓ تتعرف على آليات الوصول إلى الملفات
- ✓ تتعرف على الهيكل الشجري للأدلة



وصف الدرس

في هذا الدرس سنتناول:

- إدارة أجهزة الإدخال/الإخراج.
- وحدة التحكم بالأجهزة.
- إدخال/إخراج معنون بالذاكرة . Memory-Mapped I/O
- الوصول المباشر للذاكرة.
- إدارة الملفات.
- خصائص الملفات.
- عمليات الملفات.
- آليات الوصول للملفات.
- الدليل.
- نظرة عامة على الدليل.
- هيكلية الأدلة.



الموضوع السادس | أجهزة الإدخال والإخراج

- إحدى الوظائف المهمة لنظام التشغيل هي إدارة أجهزة الإدخال/الإخراج، بما في ذلك الفأرة، ولوحات المفاتيح، ولوحة اللمس، ومحركات الأقراص، وغيرها.
- نظام الإدخال/الإخراج ضروري لتلقي طلب إدخال/إخراج من أحد التطبيقات وإرساله إلى الجهاز المادي، ومن ثم استقبال أي استجابة من الجهاز وإرسالها مرة أخرى إلى التطبيق.
- يمكن تقسيم أجهزة الإدخال/الإخراج إلى فئتين:

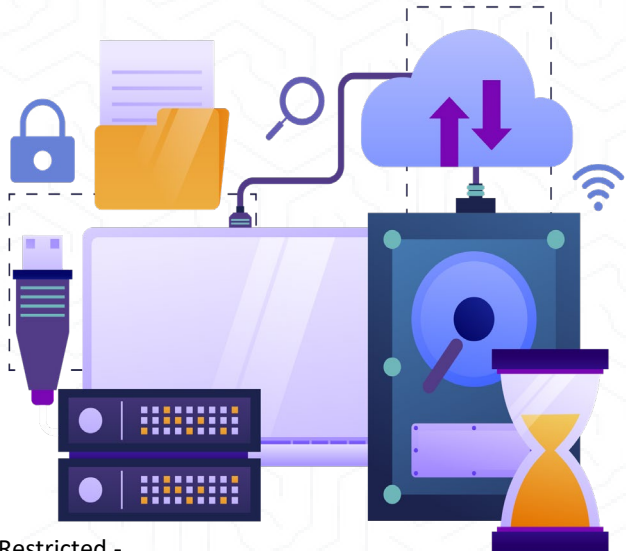
• أجهزة الكتل (Block Devices):

• هي الأجهزة التي يتواصل معها برنامج التشغيل عن طريق إرسال كتل كاملة من البيانات.

• على سبيل المثال، الأقراص الصلبة، الكاميرات المتصلة عبر USB ومحركات الأقراص المحمولة.

• الأجهزة الحرفية (Character Devices): هي الأجهزة التي يتواصل معها برنامج التشغيل عبر إرسال واستقبال أحرف فردية بشكل متسلسل.

• على سبيل المثال، المنافذ التسلسلية، المنافذ المتوازية، وبطاقات الصوت.

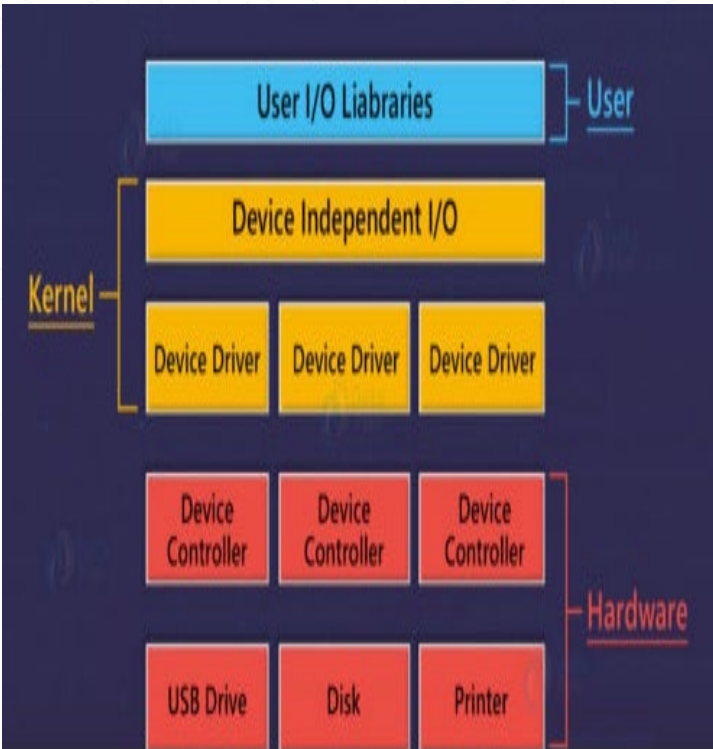


الموضوع السادس | وحدة التحكم في الأجهزة Device Controllers

وحدة التحكم في الأجهزة:

- تعتمد برامج التشغيل على كونها وحدات برمجية يتم إضافتها إلى نظام التشغيل للتعامل مع أجهزة محددة.
- يقوم نظام التشغيل بالاعتماد على برامج التشغيل لإدارة جميع أجهزة الإدخال/الإخراج.
- تعمل وحدة التحكم في الأجهزة كواجهة بين الجهاز المادي وبرنامج التشغيل الخاص بالجهاز.
- تتكون وحدات الإدخال/الإخراج (مثل لوحة المفاتيح، الفأرة، الطابعة، وغيرها) عادةً من مكونين:

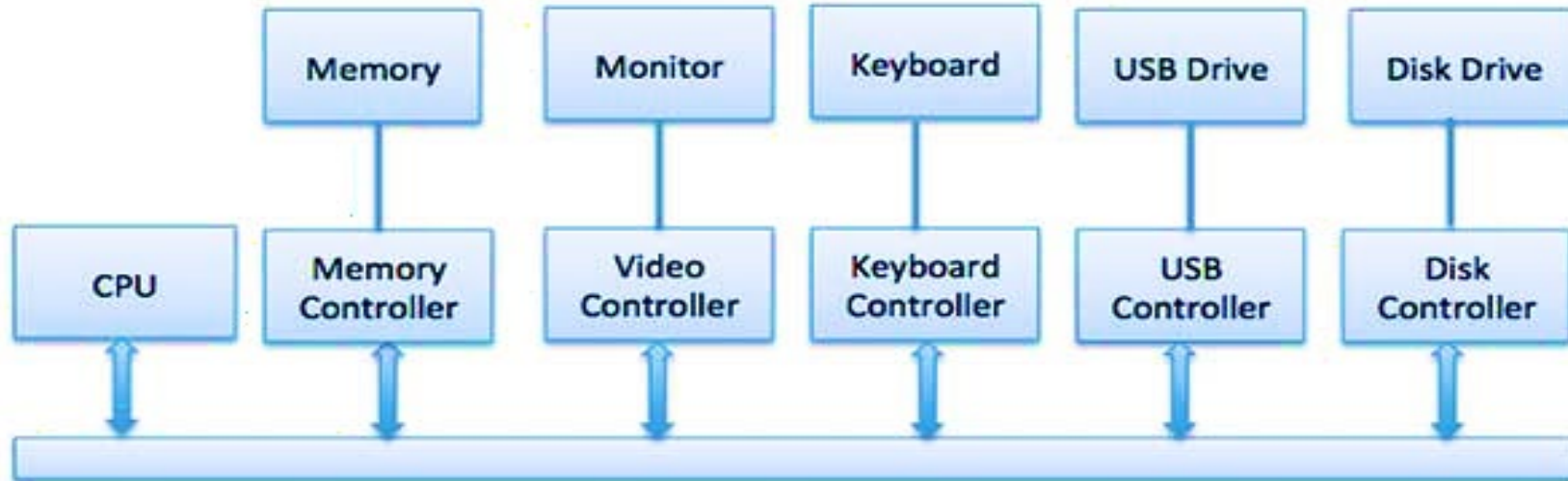
- المكون الميكانيكي: يمثل الجزء الفعلي الملموس من الجهاز.
- المكون الإلكتروني: يُعرف بوحدة التحكم في الأجهزة.



الموضوع السادس | وحدة التحكم في الأجهزة Device Controllers

وحدة التحكم في الأجهزة:

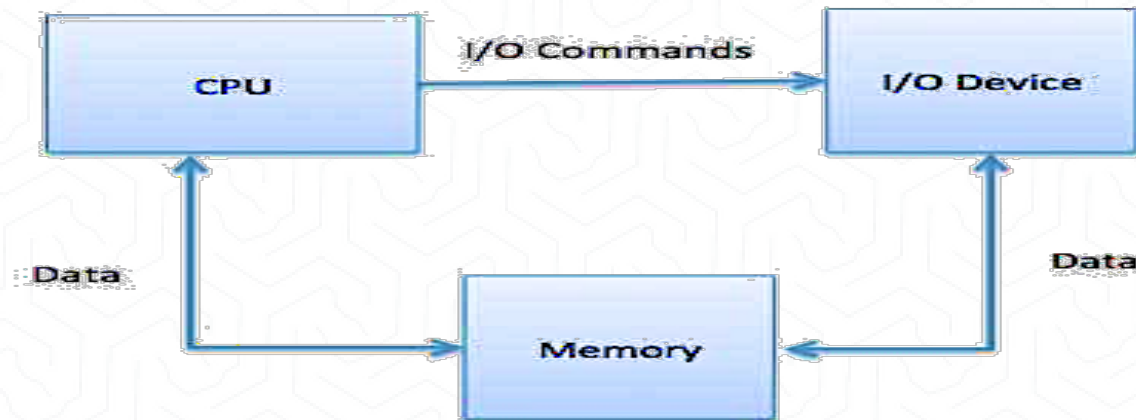
- يتوفر لكل جهاز وحدة تحكم في الأجهزة وبرنامج تشغيل خاص به للتواصل مع نظام التشغيل.
- قد تكون وحدة التحكم في الأجهزة قادرة على إدارة عدة أجهزة في وقت واحد.
- في النموذج التالي، يتم توضيح كيفية توصيل وحدة المعالجة المركزية (CPU) والذاكرة (Memory) ووحدات التحكم في الأجهزة، وأجهزة الإدخال/الإخراج (I/O Devices)، حيث تستخدم وحدة المعالجة المركزية ووحدات التحكم في الأجهزة ناقلًا مشتركًا (Common Bus) للتواصل.



الموضوع السادس | الإدخال/الإخراج المعنون بالذاكرة

الإدخال/الإخراج المعنون بالذاكرة Memory-Mapped I/O

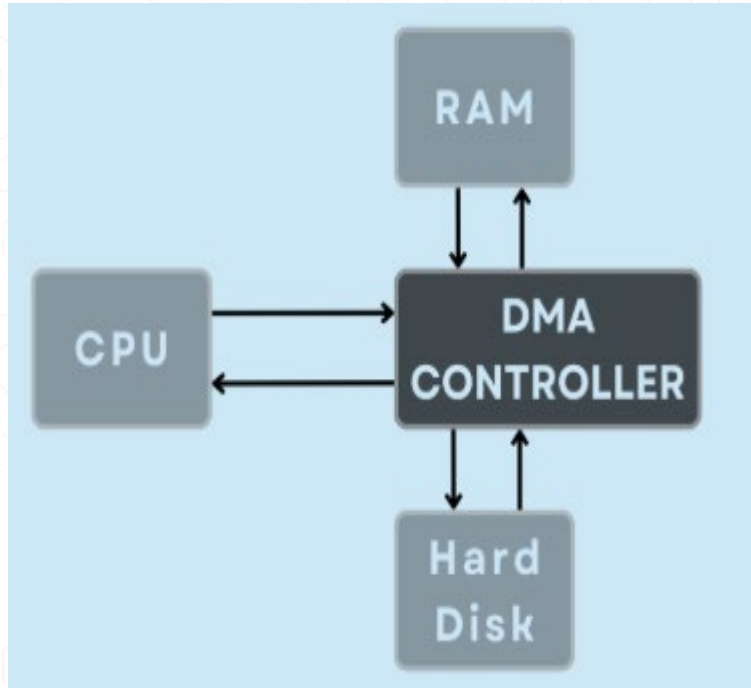
- تقنية تُستخدم لربط أجهزة الإدخال/الإخراج بالذاكرة، حيث تُعامل الأجهزة كما لو كانت مواقع ذاكرة، ويُمنح كل جهاز عنوان ضمن مساحة العناوين.
- عند استخدام هذه التقنية:
- تتم مشاركة مساحة العناوين (Address Space) بين الذاكرة وأجهزة الإدخال/الإخراج.
- يتم توصيل الجهاز مباشرة بمواقع محددة في الذاكرة الرئيسية (Main Memory)، مما يسمح بنقل البيانات (Data Blocks) مباشرة إلى/من الذاكرة دون الحاجة إلى المرور عبر وحدة المعالجة المركزية (CPU).



الموضوع السادس | الوصول المباشر الي الذاكرة DMA

الوصول المباشر الي الذاكرة Direct Memory Access

- الوصول المباشر للذاكرة هو تقنية تُستخدم لنسخ البيانات أو نقلها من موقع إلى آخر داخل النظام دون الحاجة إلى تدخل وحدة المعالجة المركزية (CPU)
- يُستخدم لتسريع عمليات نقل البيانات الكبيرة بين الأجهزة (مثل الأقراص الصلبة) والذاكرة.
- يقلل من عبء المعالجة على CPU ويحررها لتنفيذ مهام أخرى.
- تتمثل مهمة وحدة تحكم الوصول المباشر للذاكرة في تنفيذ عملية نسخ البيانات من موقع مصدر (Source Location) إلى موقع هدف (Destination Location) داخل الذاكرة أو بين الأجهزة المختلفة.
- هذه التقنية تُستخدم في أنظمة التشغيل لتحسين الأداء، خاصة عند التعامل مع كميات كبيرة من البيانات.



الموضوع السادس | إدارة الملفات File Managment

إدارة الملفات

- الملف هو وحدة منظمة من البيانات تم تسميتها وتسجيلها على وحدة التخزين الثانوية، ويُستخدم لتخزين معلومات رقمية مثل النصوص أو البرامج أو الصور أو قواعد البيانات.
- نوع الملف: يشير نوع الملف إلى قدرة نظام التشغيل على التمييز بين أنواع الملفات المختلفة مثل ملفات النصوص وملفات المصدر والملفات الثنائية، وغيرها.
- تحتوي أنظمة التشغيل مثل MS-DOS و UNIX على الأنواع التالية من الملفات:

الملفات العادية:

- هذه هي الملفات التي تحتوي على معلومات المستخدم. قد تحتوي هذه الملفات على نصوص أو قواعد بيانات أو برامج تنفيذية.

ملفات الدليل:

- تحتوي على قائمة بأسماء الملفات ومعلومات تتعلق بها، وتُستخدم لتنظيم الملفات في هيكل شجري أو هرمي داخل نظام التشغيل.

الموضوع السادس | إدارة الملفات File Management

هيكلية الملفات وأنواعها

كل نوع من الملفات يُنظم وفق هيكل معين بناءً على طبيعة البيانات التي يحتوي عليها. من الأمثلة على الملفات:

- **ملف نصي (Text File)**

يحتوي على تسلسل من الحروف (characters) تُنظم في سطور. يُستخدم لحفظ النصوص العادية.

- **ملف المصدر (Source File)**

يتكون من تعليمات مكتوبة بلغة برمجة، ويحتوي على روتينات ودوال تتبعها أوامر قابلة للتنفيذ عند الترجمة أو التفسير.

- **ملف الكائن (Object File)**

هو ناتج عملية الترجمة (compilation)، ويحتوي على بيانات منظمة تُستخدم بواسطة نظام التشغيل. عادة ما تكون هذه الملفات غير قابلة للتنفيذ مباشرة.

- **ملف تنفيذي (Executable File)**

يحتوي على كود جاهز للتنفيذ، ويمكن تحميله مباشرة إلى الذاكرة وتشغيله بواسطة وحدة التحميل (Loader). يتضمن أقسام الشيفرة والبيانات وأقسام أخرى.

الموضوع السادس | خصائص الملفات File Attributes

خصائص الملف:

الملف يُسمى لراحة المستخدمين، ويُشار إليه بواسطة اسمه. الاسم عادة ما يكون سلسلة من الحروف، مثل: example.c

• خصائص الملف:

تختلف خصائص الملف من نظام تشغيل لآخر، ولكن عادة ما تتكوّن من:

• الاسم: اسم الملف الرمزي هو المعلومات الوحيدة التي يتم الاحتفاظ بها بصيغة قابلة للقراءة من قبل البشر.

• المُعرّف: هذه العلامة الفريدة، عادة ما تكون رقمًا، تُحدد الملف داخل نظام الملفات؛ وهي الاسم غير القابل للقراءة من قبل البشر للملف.

• النوع: هذه المعلومات مطلوبة للأنظمة التي تدعم أنواعًا مختلفة من الملفات.

الموضوع السادس | خصائص الملفات File Attributes

خصائص الملف:

- الموقع: هذه المعلومات هي مؤشر إلى الجهاز وإلى موقع الملف على ذلك الجهاز.
- الحجم: الحجم الحالي للملف (بالبايتات أو الكلمات أو الكتل) وربما الحجم الأقصى المسموح به يتم تضمينه في هذه الخاصية.
- الحماية: تحدد معلومات التحكم في الوصول من يمكنه إجراء العمليات مثل القراءة، الكتابة، التنفيذ، وما إلى ذلك.
- الوقت والتاريخ ومعرف المستخدم: قد يتم الاحتفاظ بهذه المعلومات لتاريخ الإنشاء، آخر تعديل، وآخر استخدام.
- يمكن أن تكون هذه البيانات مفيدة للحماية، والأمان، ومراقبة الاستخدام.

الموضوع السادس | عمليات الملفات File Operations

عمليات الملف:

- الملف هو وحدة بيانات مجردة يتم التعامل معها وفقًا لنظام التشغيل. ولكي يكون تعريف الملف دقيقًا، يجب الأخذ بعين الاعتبار العمليات التي يمكن تنفيذها عليه.
- يقدم نظام التشغيل مجموعة من استدعاءات النظام (System Calls) التي تتيح تنفيذ عمليات متعددة على الملفات، منها:
 - إنشاء ملف (Create)
 - كتابة بيانات إلى الملف (Write)
 - قراءة بيانات من الملف (Read)
 - إعادة تموضع المؤشر داخل الملف (Reposition/Seek)
 - حذف الملف (Delete)
 - تقليص الحجم (Truncate)

الموضوع السادس | عمليات الملفات File Operations

عمليات الملف:

- يقوم نظام التشغيل بتنفيذ كل العمليات المذكورة سابقا عبر واجهات مخصصة لإدارة الملفات، وتُعدّ هذه الوظائف أساسية لعمل الأنظمة.

تشمل العمليات الأخرى:

- إعادة تسمية الملف (Rename)
- النسخ (Copy)
- الربط مع ملفات أخرى (Link)
- تغيير الصلاحيات أو السمات (Change attributes)
- لنقم بدراسة ما يجب على نظام التشغيل القيام به لتنفيذ كل من هذه العمليات الأساسية للملفات.
- وبعد ذلك، سيكون من السهل فهم كيفية تنفيذ عمليات مشابهة مثل إعادة تسمية الملف.

الموضوع السادس | عمليات الملفات File Operations

عمليات الملف:

- إنشاء ملف: لإنشاء ملف، أولاً، يجب عند إنشاء ملف جديد، يجب أولاً تخصيص مساحة له داخل نظام الملفات، و تخصيص سجل للملف داخل الدليل.
- كتابة ملف: لكتابة ملف، نقوم بإجراء استدعاء نظام نحدد فيه اسم الملف والمعلومات التي سيتم كتابتها في الملف.
- قراءة ملف: لقراءة البيانات من ملف، يتم استخدام استدعاء نظام آخر يحدد اسم الملف الذي نريد قراءته. بالإضافة إلى ذلك، يحدد هذا الاستدعاء مكان وضع البيانات في الذاكرة.
- إعادة تموضع المؤشر داخل الملف: في هذه العملية، يقوم نظام التشغيل بالبحث في الدليل للعثور على الإدخال المناسب للملف الذي نريد العمل عليه. بمجرد تحديد الملف، يتم إعادة تموضع المؤشر داخل الملف إلى موقع معين، أي تحديد مكان داخل الملف حيث سيبدأ النظام القراءة أو الكتابة. هذا يسمح للمستخدم أو للبرنامج بتغيير موقعه داخل الملف دون الحاجة إلى البدء من البداية

الموضوع السادس | عمليات الملفات File Operations

عمليات الملف:

- **حذف ملف**
 - يتم حذف ملف عندما لا تكون هناك حاجة إليه.
 - تتضمن هذه العملية إزالة الإدخالات الخاصة بالملف من دليل النظام، وتحرير المساحة التي كان يحتلها في وحدة التخزين.
 - يتم البحث في الدليل عن الملف المطلوب، بعد العثور على الإدخال المرتبط به في الدليل:
 - يتم تحرير جميع المساحات المخصصة للملف بحيث يمكن إعادة استخدامها من قبل ملفات أخرى.
 - يتم مسح الإدخال الخاص بالملف من الدليل.
- **تقليص أو تمديد حجم الملف**
 - يُستخدم تقليص حجم الملف عن طريق إزالة جزء أو كل البيانات المخزنة فيه، مع الاحتفاظ بخصائص الملف الأخرى مثل الاسم، الأذونات، والموقع.
- **خطوات عملية تقليص الملف:**
 1. إعادة تعيين طول الملف إلى صفر.
 2. تحرير مساحة التخزين المستخدمة مسبقًا بواسطة محتوى الملف، ليتمكن النظام من إعادة تخصيصها.
 3. الحفاظ على جميع الخصائص الأخرى للملف دون تغيير.

الموضوع السادس | آليات الوصول الى الملفات

الوصول المتسلسل (Sequential Access)

- هو الشكل الأبسط والأكثر شيوعًا.
- تتم قراءة البيانات أو كتابتها بتسلسل من البداية إلى النهاية فقط.
- لا يمكن القفز إلى موقع معين مباشرةً.
- يُستخدم عادةً في التطبيقات التي تتطلب المعالجة المتتالية للبيانات مثل ملفات النصوص.

الوصول المباشر/العشوائي (Direct/Random Access)

- يسمح بالوصول إلى أي سجل داخل الملف مباشرةً.
- يتم استخدام المؤشرات أو العناوين للوصول إلى مواقع محددة.
- يُستخدم هذا النمط مع الأقراص والأنظمة التي تتطلب سرعة وصول مثل قواعد البيانات.

الموضوع السادس | آليات الوصول الى الملفات

الوصول المتسلسل المفهرس (Indexed Sequential Access)

- هذه الآلية تجمع بين الوصول المتسلسل والوصول المباشر.
- أولاً، يتم إنشاء فهرس يحتوي على مؤشرات تشير إلى المواقع المختلفة في الملف. هذا الفهرس يُخزن معلومات حول كيفية الوصول إلى البيانات داخل الملف.
- عند البحث في الملف، يتم البحث أولاً في الفهرس بشكل متسلسل، ومن ثم يتم استخدام المؤشر الموجود في الفهرس للوصول إلى الجزء المحدد من الملف مباشرة.
- هذا النوع يوفر كفاءة عالية، حيث يتم تسريع الوصول إلى السجلات باستخدام الفهرس، بينما يظل من الممكن الوصول إلى البيانات بشكل تسلسلي إذا لزم الأمر.

الموضوع السادس | الدليل Directory

- الدليل هو بنية بيانات تُستخدم لتنظيم الملفات في أنظمة التشغيل، ويُعتبر عنصرًا أساسيًا في نظام الملفات.
- يوجد في أنظمة التشغيل المختلفة مثل: Unix, OS/2, MS-DOS, وLinux.
- يُمثل الدليل جدولًا رمزيًا يُترجم أسماء الملفات إلى معلومات موقعها الفيزيائي على وحدة التخزين.

يحتوي كل إدخال في الدليل عادة على:

- اسم الملف.
- موقع الملف على القرص.
- نوع الملف أو امتداده.
- بعض السمات (مثل التاريخ، الحجم، الامتيازات).
- مؤشر للوصول (في بعض التصاميم).

الموضوع السادس | الدليل Directory

العمليات الأساسية على الدليل:

- إضافة مدخل: عند إنشاء ملف جديد، تتم إضافة اسم الملف وموقعه إلى الدليل.
- حذف مدخل: عند حذف ملف، يتم حذف مدخله من الدليل.
- البحث عن ملف: يتم تحديد موقع الملف من خلال اسمه.
- عرض محتويات الدليل: استرجاع أسماء جميع الملفات والمجلدات الفرعية الموجودة ضمن الدليل.

الموضوع السادس | الدليل Directory

البحث عن ملف:

- القدرة على البحث ضمن هيكل الدليل للعثور على الإدخال الخاص بملف معين.
- إمكانية البحث عن الملفات التي تتطابق أسماؤها مع نمط معين.
- إنشاء ملف: إنشاء ملفات جديدة وإضافتها إلى الدليل.
- حذف ملف: إزالة الملفات التي لم تعد بحاجة إليها من الدليل.
- عرض محتويات الدليل: عرض الملفات في الدليل ومحتويات إدخال الدليل لكل ملف.

إعادة تسمية ملف:

- تغيير اسم الملف عند تغيير محتواه أو استخدامه.
- إمكانية تغيير موقع الملف ضمن هيكل الدليل.
- التنقل عبر نظام الملفات: الوصول إلى كل دليل وكل ملف داخل هيكل الدليل.

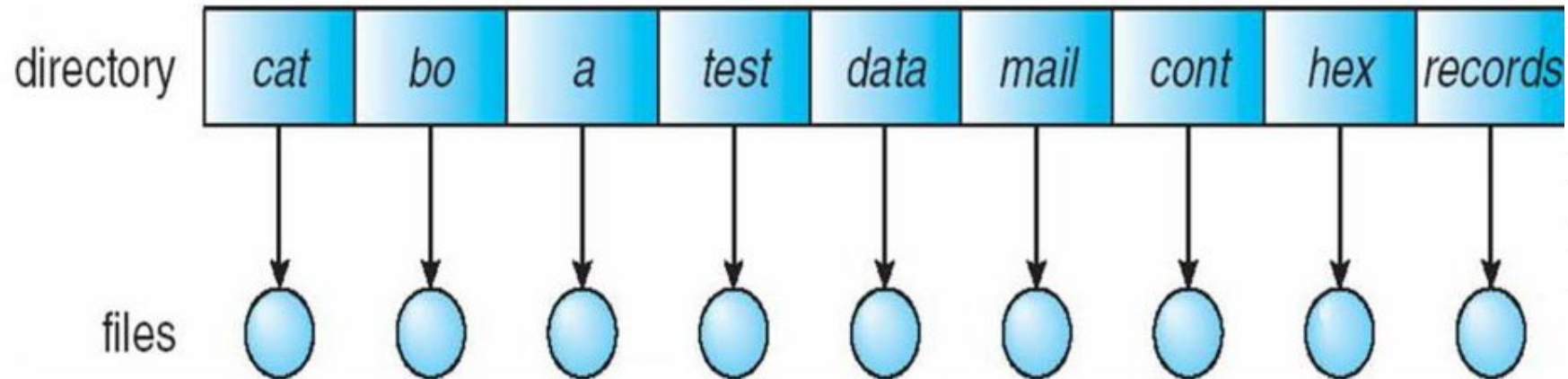
الموضوع السادس | هيكلية الأدلة Directory Structures

الدليل ذو المستوى الواحد (Single-Level Directory):

- أبسط هيكل دليل هي الدليل ذو المستوى الواحد، حيث يتم تخزين جميع الملفات ضمن دليل واحد مشترك.
- المميزات: هذا الترتيب بسيط وسهل الفهم والصيانة، كما يسهّل العثور على الملفات حيث لا يوجد إلا موقع واحد للبحث.

القيود:

- لا يسمح بوجود ملفات بأسماء متشابهة لمستخدمين مختلفين، مما يتسبب في تضارب الأسماء.
- عند زيادة عدد الملفات، تصبح عملية التنظيم أكثر صعوبة.
- لا يدعم تنظيم الملفات بحسب المستخدم أو نوع التطبيق، ما يؤدي إلى تشويش وازدحام في هيكل الدليل.



الموضوع السادس | هيكلية الأدلة Directory Structures

الدليل ذو المستويين (Two Level Directory)

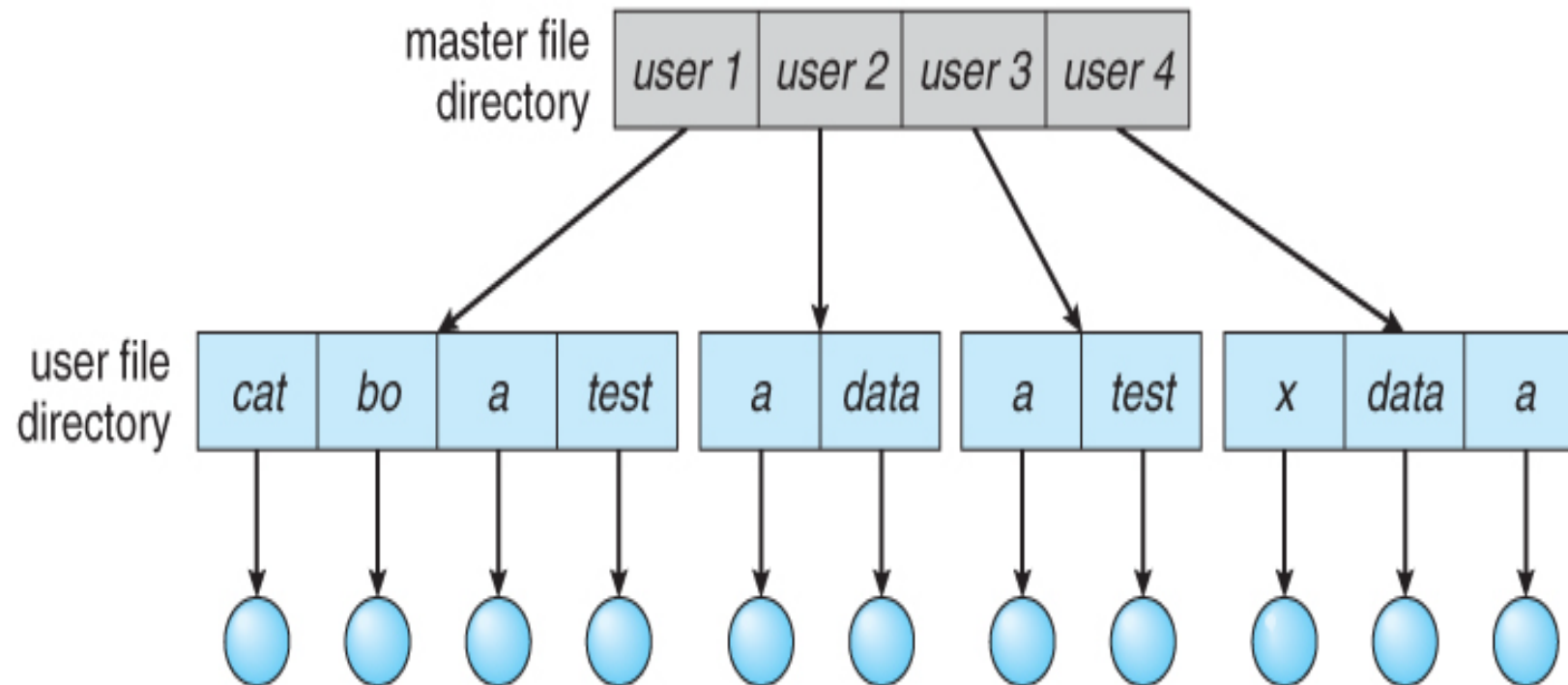
- في الدليل ذو المستوى الواحد، قد يحدث تداخل في أسماء الملفات بين المستخدمين المختلفين.
- الحل القياسي: إنشاء دليل منفصل لكل مستخدم.

في هيكلية الدليل ذو المستويين:

- يحتوي النظام على دليل رئيسي (Master File Directory - MFD) يحتوي على إشارات لكل مستخدم.
- كل مستخدم يمتلك دليل ملفات مستخدم (User File Directory - UFD) خاص به.
- يسمح هذا النموذج: بفصل ملفات المستخدمين، لكنه لا يدعم المشاركة بين المستخدمين، لأن كل مستخدم يرى الملفات التي أنشأها فقط.

الموضوع السادس | هيكلية الأدلة Directory Structures

- الدليل ذو المستويين (Two Level Directory)



الموضوع السادس | هيكلية الأدلة Directory Structures

الدليل ذو الهيكل الشجري (Tree Structured Directories)

- بعد فهم بنية الدليل ذي المستويين، يمكن تمثيل الأدلة باستخدام بنية شجرية أكثر مرونة.
- في هذا الهيكل، يصبح الدليل الرئيسي (root) نقطة البداية وتتشعب منه الأدلة الفرعية.
- يسمح هذا التنظيم بإنشاء دلائل خاصة لكل مستخدم أو تطبيق أو قسم، مما يسهّل إدارة الملفات.
- يمكن للمستخدمين إنشاء فروع خاصة بهم لتنظيم ملفاتهم بمرونة وكفاءة أعلى.
- يعد هذا النموذج الأكثر شيوعًا في أنظمة التشغيل الحديثة.
- يحتوي على دليل الجذر (Root Directory) الذي يتفرع إلى أدلة فرعية تمثل المستخدمين أو التطبيقات.
- كل ملف داخل النظام له مسار فريد (Unique Path Name) يبدأ من الدليل الجذري.

الموضوع السادس | هيكلية الأدلة Directory Structures

مزايا الدليل ذو الهيكل الشجري (Tree Structured Directories)

- تنظيم هرمي واضح وسهل الفهم.
- يتيح إمكانية تكرار أسماء الملفات في دلائل مختلفة.
- يسهل تنفيذ صلاحيات الوصول والتفويض لكل دليل أو ملف.

